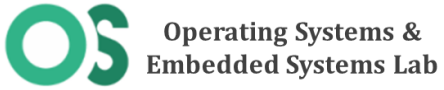
Operating Systems Lab 2
Shell Program

Due date: 2026/10/30 17:00 (before lab2 course finishes)

oslab@mail.csie.ncku.edu.tw

TA: ne6144074@gs.ncku.edu.tw、p76144728@gs.ncku.edu.tw、p76154537@gs.ncku.edu.tw



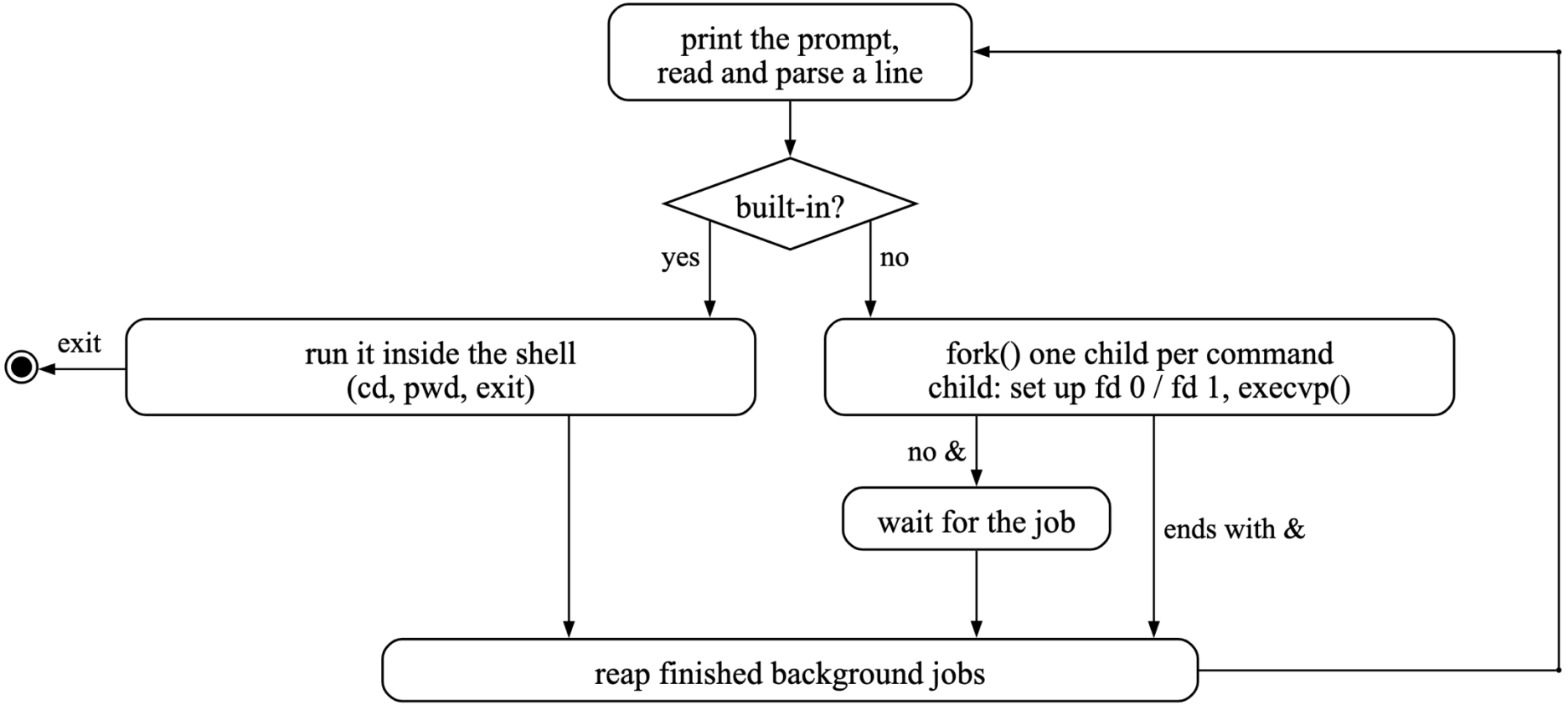


Outline

- Overview
- Requirement & Flow
- Related Works
- Demo & Grading
- Precautions

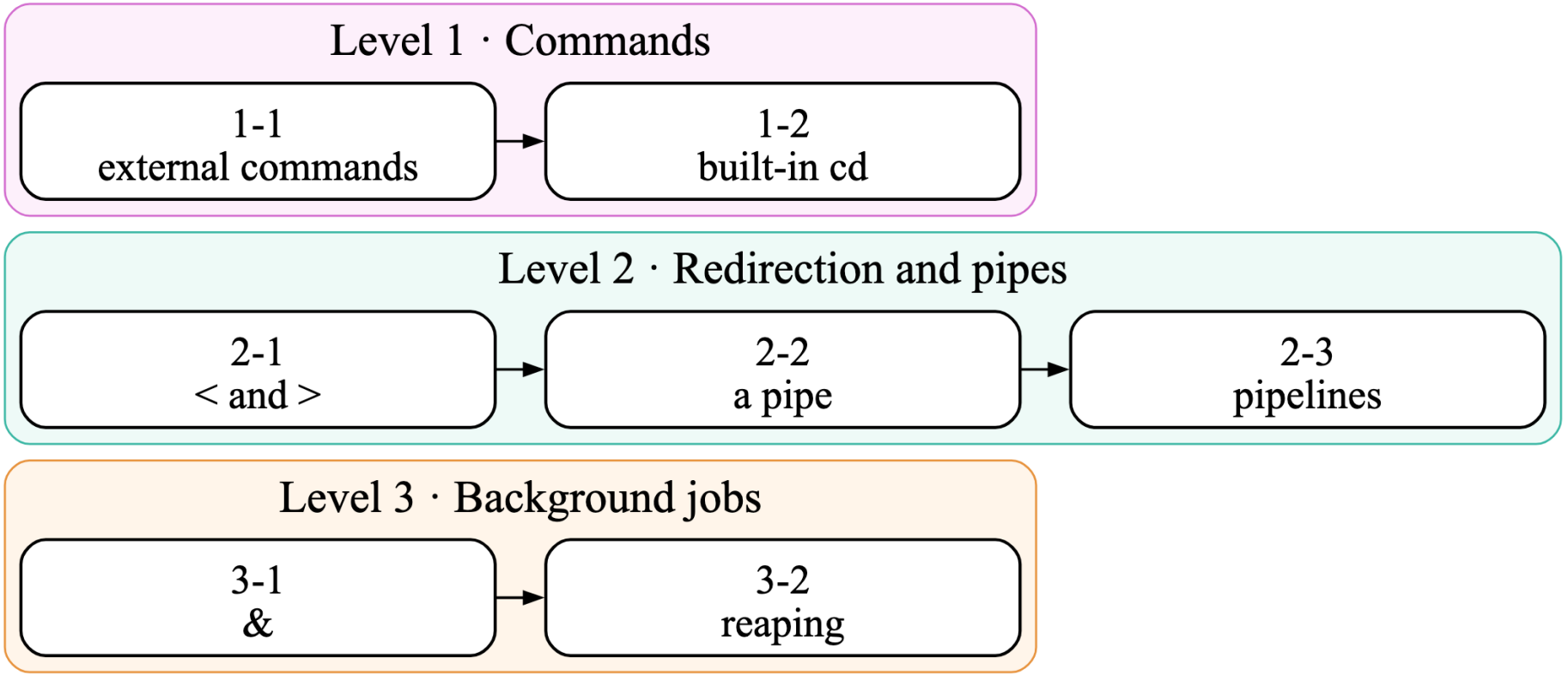
Overview: What Is a Shell?

- A shell is a program that **runs other programs** for you: `ls`, `cat`, `grep` ...
- In this lab you finish a shell program, `my_shell`



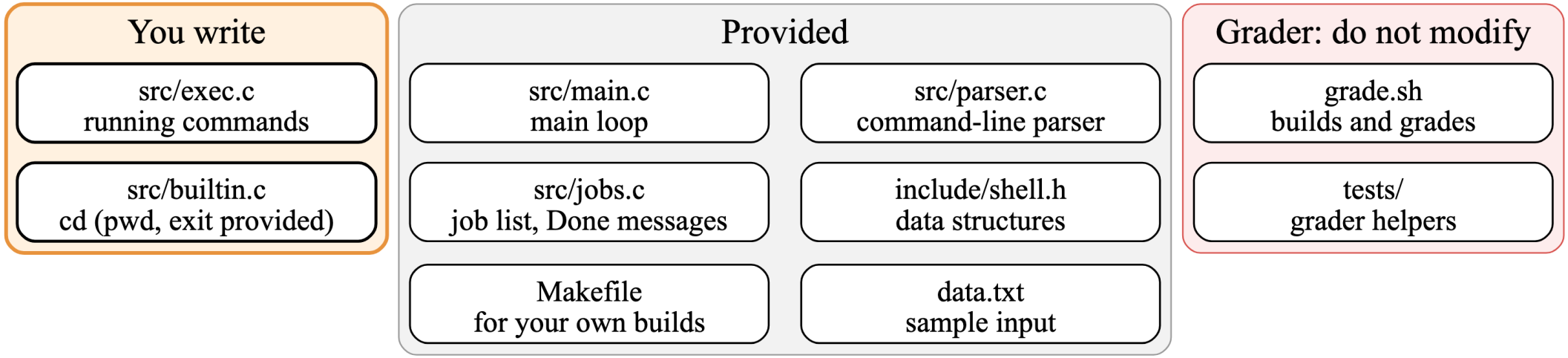
Overview: Seven Steps, Three Levels

- Each step builds on the one before it: do them **in order**



Overview: Template

- Every TODO is tagged with its step: `/* TODO [1-1] ... */`
- Find the work for one step: `grep -n "TODO \[2-2\]" src/*.c`
- Keep every line marked (`provided`)
- Template: https://github.com/geumbo/os_2026_lab2_template



Requirement & Flow: Input and Output

- Input: `cmd args [< in] [> out] | cmd args ... [&]`
- Words are separated by spaces; `&` only at the end;
  built-ins (`cd`, `pwd`, `exit`) take no `| < > &`
- The grader checks the output **exactly**. Full rules: README sections 4 and 5

```
>>> $ sort < data.txt > sorted.txt      ← < and >: read and write files
>>> $ cat data.txt | wc -l              ← |: connect two commands
8
>>> $ sleep 2 &                         ← &: run in the background
[1] 59750                               ← [job number] pid
>>> $ nosuchcmd
nosuchcmd: command not found
>>> $                                   ← press Enter after 2 seconds
[1]  Done       sleep 2 &               ← printed before the next prompt
>>> $
```

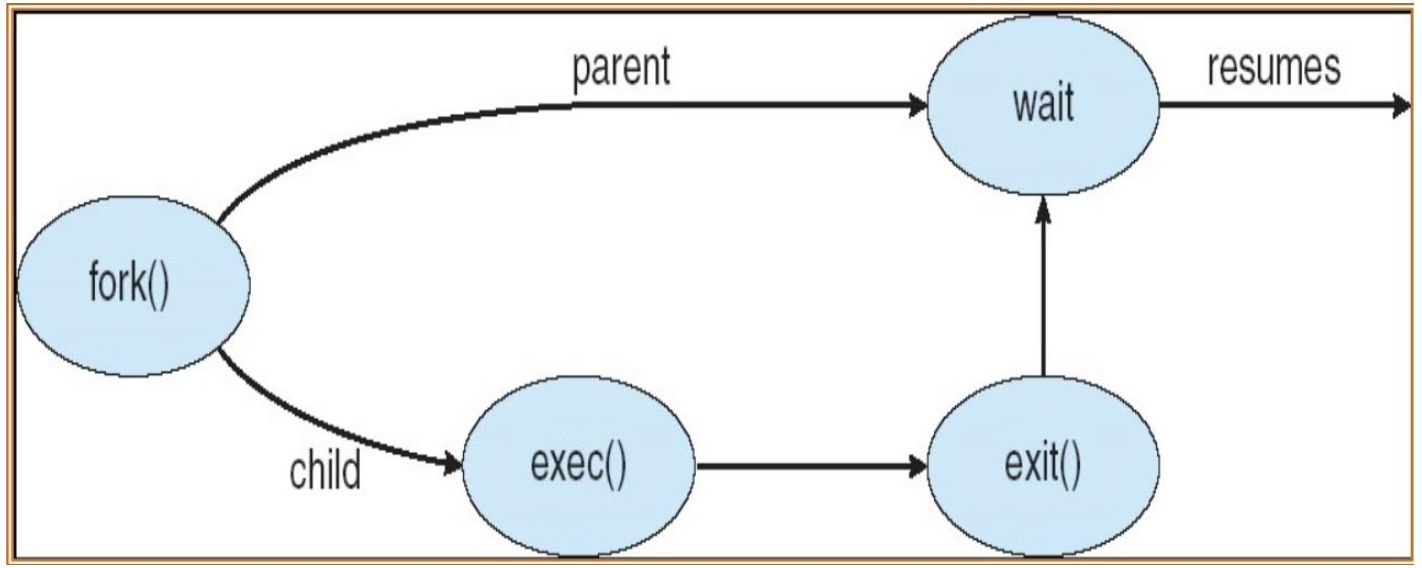
Requirement & Flow: `$?`

- `$?` is the exit status of the last command. Full table: README section 5

```
>>> $ false
>>> $ echo $?
1                          ← the command's exit code
>>> $ true | false
>>> $ echo $?
1                          ← a pipeline: its last command
>>> $ nosuchcmd
nosuchcmd: command not found
>>> $ echo $?
127                        ← command not found
>>> $ cat < missing.txt
missing.txt: No such file or directory
>>> $ echo $?
1                          ← a redirection file could not be opened
>>> $ sleep 5 &
[1] 59798
>>> $ echo $?
0                          ← started in the background
```

1-1 External Commands

- `fork()` a child → the child `execvp()`s the program → the shell `waitpid()`s for it
- `$?` = `exit_code(status)`: `WIFEXITED` → `WEXITSTATUS`, otherwise 128 + `WTERMSIG`
- If `execvp()` returns, it failed: print `<cmd>: command not found`, `_exit(127)`
- In the child use `_exit()`, not `exit()`: `exit()` would print the shell's unflushed output a second time
- Functions: `exit_code()`, `launch_job()`, `launch_process()`, `wait_for_job()` in `src/exec.c`



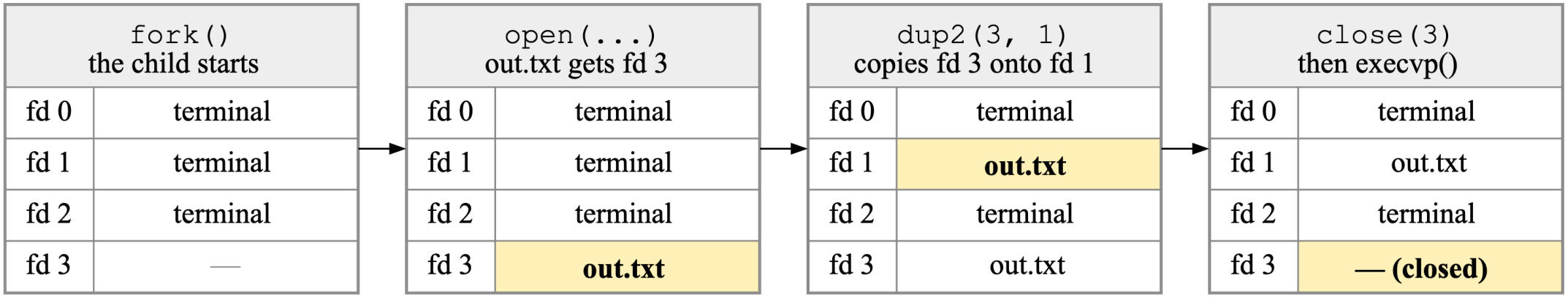
1-2 Built-in Command: `cd`

- Try `cd /tmp` after 1-1: **nothing changes**. Why?
  - A child has its own PCB and its own working directory
  - `chdir()` in a child does not move the parent
- So `cd` must run **inside the shell**, without `fork()`
- cd with no argument goes to `$HOME`; on failure `perror("cd")`, `$? = 1`
- Function: `builtin_cd()` in `src/builtin.c`

```
>>> $ cd /tmp
>>> $ pwd
/tmp
>>> $ cd /no_such_dir
cd: No such file or directory
>>> $ echo $?
1
```

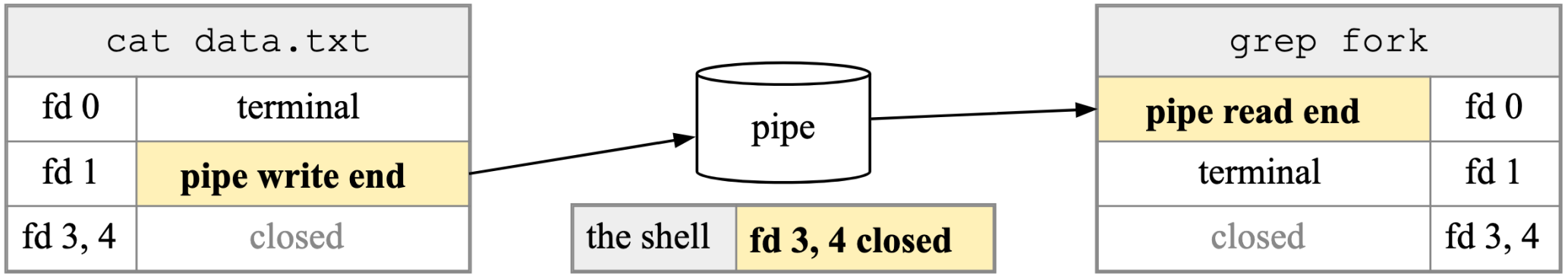
2-1 Redirection

- Between `fork()` and `exec`, the child replaces fd 0 or fd 1 with a file
- `open()` the file → `dup2()` it onto fd 0 or fd 1 → `close()` the original
- The program writes to fd 1 as usual and **never knows** it is a file
- The shell's own fds never change: **only the child** redirects
- `open()` flags: see slide 18



2-2 A Pipe

- Connect two commands: `cat data.txt | grep fork`
- The shell creates the pipe **before `fork()`**, so both children inherit it
- Each child `dup2()`s its end onto fd 0 or fd 1
- Every process closes the pipe ends it does not use, including the shell
- Connect the pipe ends first, then < and >: **a file wins** (`echo pipe > diverted.txt` | `cat` prints nothing)
- `pipe()` returned fd 3 (read end, `mypipe[0]`) and fd 4 (write end, `mypipe[1]`); the numbers can differ



2-2 Why Close Every Unused End?

- A reader sees end-of-file only when **every write end** is closed
- A writer gets SIGPIPE when it **writes** to a pipe whose read ends are all closed
- Try it in bash:

```
$ sleep 5 | wc -l       # wc waits 5 s: sleep still holds the write end
0
```

Missing close	What happens
the shell keeps the write end	grep never sees end-of-file and waits forever, and so does the shell
cat keeps the read end	when the reader exits, the writer gets no SIGPIPE: yes | head -n 1 never ends

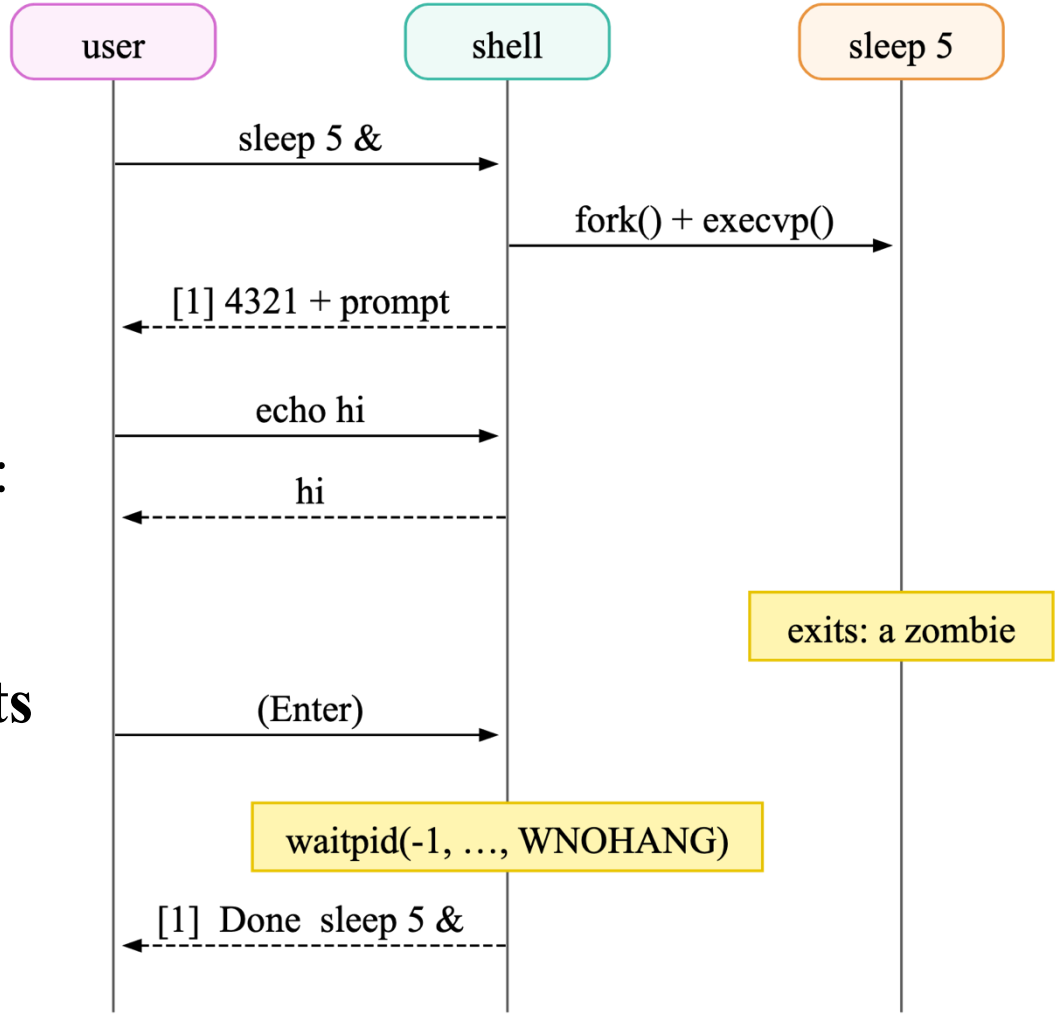
2-3 Pipelines

- Any number of commands: one pipe per `|`
- Fork every process **first**, **then wait**: they all run at the same time
- A pipe holds a limited amount of data (usually 64 KiB on Linux); a writer to a full pipe waits: waiting for `cat` before starting `wc` can block forever
- Wait for **every process**, even if the last one ends first (`sleep 1 | echo hi` waits 1 s)
- `$?` of a pipeline = the status of its **last process**
- Function: `wait_for_job()`. If your 2-2 loop is general, nothing else is new

```
>>> $ cat data.txt | grep fork | wc -l
3
>>> $ seq 1 100000 | cat | tail -n 1
100000
```

3-1 Background Jobs

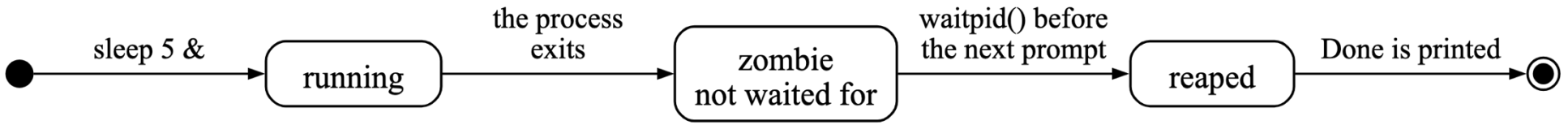
- A line ending with `&` runs while the shell keeps reading commands
- **Do not wait**: print `[jid] pid`, set `$?` to 0, return
- A background job must not read the terminal: its stdin becomes `/dev/null` (unless it already reads a pipe or a < file)
- A foreground command still waits **only for its own pids**: `waitpid(-1, ...)` or `wait()` would also collect background jobs



3-2 Reaping

- A finished child stays a **zombie** until its parent waits for it
- Before every prompt, collect every finished child **without blocking**:
  - loop `waitpid(-1, &status, WNOHANG)` until it returns `0` or `-1`
- The provided code then prints `Done` and removes the job
- So Done appears only when the **next prompt** is printed, not the moment the job ends
- Function: `update_bg_jobs()` in `src/exec.c`

```
>>> $ sleep 1 &
[1] 33375
>>> $ sleep 1 &
[2] 33376
>>> $                              ← press Enter after 1 second
[1]  Done       sleep 1 &
[2]  Done       sleep 1 &
```



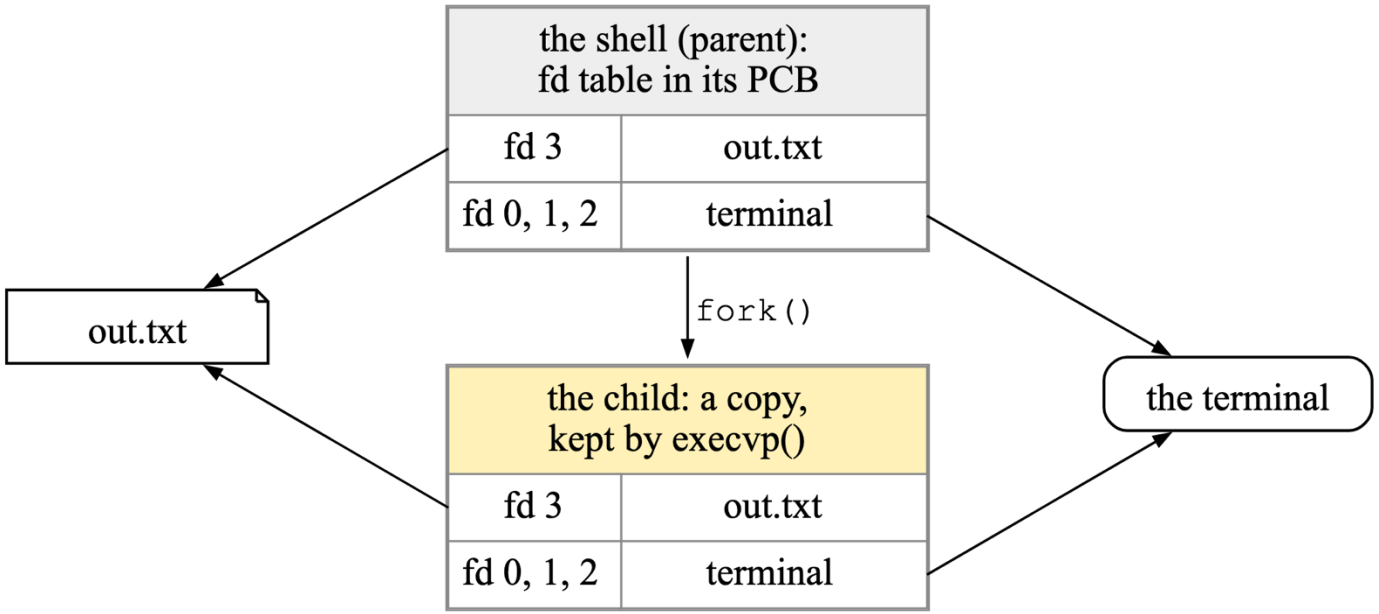
Related Works: APIs

API	What it does
`fork()`	creates a child; returns 0 in the child, the child's pid in the parent
`execvp(file, argv)`	replaces the process with a program; returns only on failure
`waitpid(pid, &st, opt)`	waits for a child; -1 = any child, WNOHANG = do not block
`WEXITSTATUS(st)`	the exit code stored in st
`_exit(code)`	ends the process at once (no stdio flush)

API	What it does
`chdir(path)`	changes the working directory of this process
`pipe(fd)`	fd[0] read end, fd[1] write end
`dup2(oldfd, newfd)`	makes newfd refer to the same file as oldfd
`open(path, flags, mode)`	opens a file, returns a new fd
`close(fd)`	closes fd

Related Works: File Descriptors

- An fd is an index into the process's table of open files (kept in the PCB)
- fd 0 = stdin, fd 1 = stdout, fd 2 = stderr; a new fd gets the **lowest free number**
- `fork()` **copies** the table: the child's fds refer to the same files and pipes
- `exec` **keeps** the table: the new program starts with the fds the child set up
- So a shell redirects a program by changing `fd 0 / fd 1` **after `fork()` and before `exec`**



Related Works: `open()` (Supplement)

- You need `open()` for < and >:

```c
#include <fcntl.h>

int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
/* O_WRONLY  write only                          */
/* O_CREAT   create it if missing (mode 0644)    */
/* O_TRUNC   empty it if it already exists       */
/* for <, open with O_RDONLY (read only)         */
if (fd < 0) {
    perror("out.txt");   /* e.g. "out.txt: Permission denied" */
    _exit(1);
}
dup2(fd, STDOUT_FILENO); /* fd 1 now refers to out.txt */
close(fd);               /* fd 1 keeps the file open */
```

Testing Your Shell

- **Linux only**: Ubuntu 24.04 (VM, WSL2 or container), `gcc`, `python3`
- `grade.sh` builds with its own command, **not your Makefile**

```
make                          # build with your Makefile while developing
./my_shell                    # try it by hand
./grade.sh                    # run all 42 checks
./grade.sh -v                 # list every check
GRADER_ONLY=2-2 ./grade.sh    # run only the checks of one step
```

Testing Your Shell

Level	Step	Requirement	Checks	Score
1 Commands	1-1 1-2	External commands Built-in commands	5/5 5/5	✔ 1 ✔ 1
2 Redirection and pipes	2-1 2-2 2-3	Redirection A pipe Pipelines	7/7 7/7 7/7	✔ 1 ✔ 1 ✔ 1
3 Background jobs	3-1 3-2	Background jobs Reaping	6/6 5/5	✔ 1 ✔ 1
		Total	42/42	7 / 7

```
All checks passed.
Details    ./grade.sh -v lists every check
Checksum   df332e53a95e  ✔ matches README
```

Testing: When a Check Fails

- Each failed check shows what was typed, what happened, and a hint
- The tests are fixed: run the grader as often as you like
- Fix the **first failing step** first; later steps often fail for the same reason

```
✘ 2-2 A pipe  ·  6 of 7 checks failed
  ✘ the reader sees end-of-file
    typed   cat data.txt | cat
    result  did not finish within 2 s
    hint    the shell must close its copy of the write end, or the reader
            waits forever

  Next step  2-2 A pipe → TODO [2-2] in src/
```

Demo & Grading

- (7pt) Program: 7 requirements, 1pt each
  - A requirement earns its point only if all of its checks pass
  - Level 1: 1-1, 1-2 · Level 2: 2-1, 2-2, 2-3 · Level 3: 3-1, 3-2

- (3pt) Handwritten quiz (closed book, on demo day)

- Demo: run `./grade.sh` on your own machine in front of the TA
  - The TA checks the checksum on its last line: `df332e53a95e`
  - The TA may also look at your code

Precautions

- Due date: 2026/10/30 17:00 (before lab2 course finishes)
- Implement in **C**. Your work goes in `src/exec.c` and `builtin_cd()` in `src/builtin.c`
- `system()`, `popen()` and running commands through another shell (`bash -c`, `sh -c`) are not allowed: your shell must `fork()` and `exec` every command itself
- Do not modify `grade.sh` or `tests/`
- You may use any reference, including LLMs: the quiz asks about the concepts and **your own code**
- Not in this lab: `fg`, `bg`, `jobs`, Ctrl-Z, quotes, `>>`, `2>`